

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.5 Working in Background

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Receptores de emisiones**
2. Servicios
3. Comunicaciones con servicios
4. Tareas en segundo plano

Receptores de emisiones

- Las apps de Android pueden enviar o recibir mensajes de emisión desde el sistema u otras apps de forma similar al patrón publicación-suscripción
- Las emisiones se envían cuando ocurre un evento de interés:
  - batería baja, nuevo SMS, nueva wifi, conexión USB, usuario conectado
- Existen emisiones “estándar” (del sistema) y las personalizadas (propia app)
- Pueden servir para enviar información entre componentes, pero no hay que abusar
- Las emisiones se pueden recibir aunque la app no esté en ejecución o incluso nunca se haya ejecutado (aunque hay excepciones a éste último caso)

Receptores de emisiones

- Los receptores no deben ejecutar tareas prolongadas, ellos sólo “gestionan” el evento, actualizan información interna de la app y/o lanzan nuevos componentes
- Los receptores pueden ser locales (atributo *android:exported = “false”*) y pueden requerir permisos
- Hay dos tipos de receptores de emisiones:
  - Declarados en el manifiesto: siempre activos
    - (existen restricciones y desde la API 26 ya no están permitidos)
  - Registrados en el contexto: sólo funcionan si la app está activa o minimizada

Receptores declarados en el manifiesto

- Declaración en el manifiesto (pueden ser varios) y creación de la subclase

```
<receiver android:name=".MyBroadcastReceiver"  android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
        <action android:name="android.intent.action.USER_PRESENT" />
    </intent-filter>
</receiver>
```

```
public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        String log = "Action: " + intent.getAction() + "\n";
        log = log + "URI: " + intent.toUri(Intent.URI_INTENT_SCHEME) + "\n";
        Toast.makeText(context, log, Toast.LENGTH_SHORT).show();
        Log.d("_TAG", log);
    }
}
```

Receptores registrados en el contexto

1. Crea una instancia del receptor

```
BroadcastReceiver br = new MyBroadcastReceiver();
```

2. Crea un IntentFilter y registra dicho receptor con *registerReceiver*().
   a. Si el receptor se declara en la actividad, tiene acceso a sus variables

```
IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
filter.addAction(Intent.ACTION_AIRPLANE_MODE_CHANGED);
registerReceiver(br, filter);
```

3. **Muy importante**: hay que anular el registro con *unregisterReceiver*() en la función "simétrica" del ciclo de vida:

```
unregisterReceiver(br);
```

Envío de emisiones

- Existen 3 formas de enviar emisiones:
  1. A un receptor cada vez (por prioridad) y éste decide si pasar la emisión al siguiente o cortar la propagación: *sendOrderedBroadcast*(Intent, String)
  2. A todos los receptores (sin ningún orden): *sendBroadcast*(Intent)
  3. Sólo locales (eficiente y segura): LocalBroadcastManager.sendBroadcast(Intent)

Envío de emisiones propias

- Ejemplo de envío de una notificación propia y cómo se debe registrar en receptor:

```
Intent intent = new Intent();
intent.setAction("com.example.broadcast.MY_NOTIFICATION");
intent.putExtra("data", "Mi propio tipo de notificación");
sendBroadcast(intent);
```

```
IntentFilter filter = new IntentFilter("com.example.broadcast.MY_NOTIFICATION");
registerReceiver(br, filter);
```

Índice

1. Receptores de emisiones
2. **Servicios**
3. Comunicaciones con servicios
4. Tareas en segundo plano

Servicios

- Un servicio es un componente de la app que se ejecuta sin interfaz de usuario
  - Se sigue ejecutando aunque el usuario cambie de app
  - Sirve también para que una app exponga una funcionalidad a otras apps
- Se ejecuta siempre en el hilo principal de la app: no debe ejecutar tareas costosas
  - Para tareas largas o costosas: crear hilo secundario, IntentService o similar
  - No es un proceso separado ni un hilo independiente
- Como toda componente debe declararse en el manifiesto

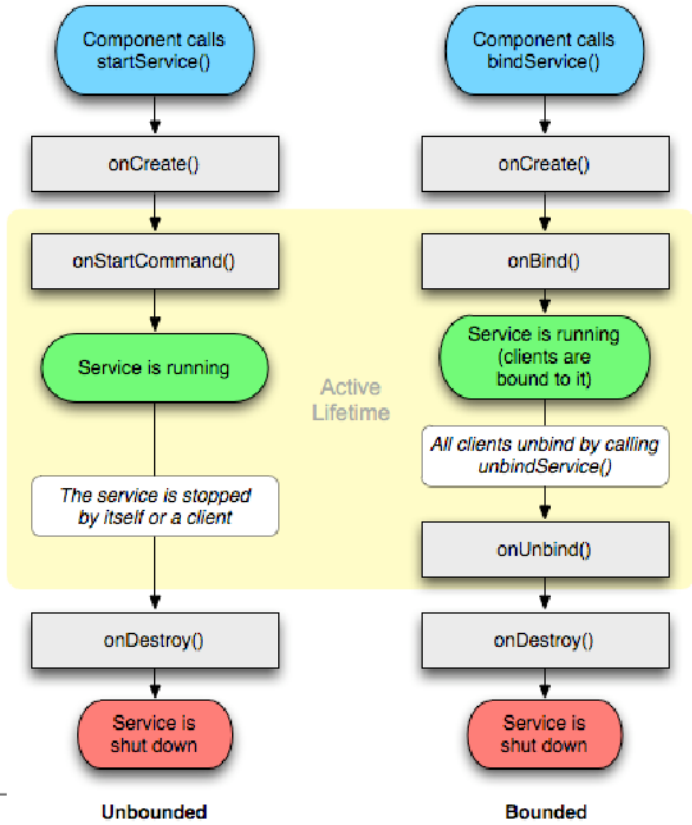
```
<service android:name=".MyService" >
</service>
```

Servicios

- Hay tres tipos de servicios (aunque no son incompatibles entre si):
  - Primer plano: asociado a una notificación para que el usuario esté informado
    i. Se verá más adelante después de explicar las notificaciones
  - Segundo plano o local: una vez lanzado (*startService*) se pierde "la conexión"
    i. Se debe finalizar una vez acabada su tarea (*stopSelf*)
    ii. También se puede finalizar externamente (*stopService*)
  - Enlazado: incluye una vinculación para una comunicación bidireccional
    i. Los componentes se pueden ir vinculando (*bindService*) y desvinculando (*unbindService*) dinámicamente durante la ejecución del servicio
    ii. Se destruye cuando no hay clientes enlazados con el servicio

Ciclo de vida de los servicios

- El ciclo de vida de un servicio depende de su tipo (aunque no son incompatibles)
- Métodos comunes a ambos tipos (ciclos):
  - onCreate(): configuración inicial
  - onDestroy(): liberación de los recursos



Servicio local: declaración y ejecución

```
public class MyService extends Service {
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        super.onStartCommand(intent, flags, startId);
        String log = " MyService ha sido ejecutado";  Log.d ("_TAG", log);
        Toast.makeText(getApplicationContext(), log, Toast.LENGTH_SHORT).show();
        return START_STICKY;
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

```
Intent service = new Intent(getApplicationContext(), MyService.class);
startService(service);
```

Servicio local: gestión reinicio

- *onStartCommand* puede devolver tres tipos de constantes al finalizar el servicio:
  - START_NON_STICKY: el sistema NO vuelve a crearlo (la opción más segura)
  - START_STICKY: el sistema vuelve a crearlo pero NO se entrega el último intent
  - START_REDELIVER_INTENT: se vuelve a crear el servicio y con el último intent (servicios que siempre ejecutan trabajos y deben reiniciarse rápido)
- En todos los casos, antes de destruir un servicio siempre se ejecutan los intents pendientes de entregar (por defecto, los servicios sólo ejecutan una tarea cada vez)

Servicio local: tareas pesadas

- Si el servicio debe ejecutar una tarea pesada el sistema falla, por ejemplo

```
// tarea pesada
try {
   Thread.sleep(5000);
} catch (InterruptedException e) {
   e.printStackTrace();
   Thread.currentThread().interrupt();
}
```

- Para ejecutar tareas largas o pesadas existen 2 opciones básicas:
  - Emplear una **IntentService**: simple (pero sólo una tarea cada vez)
  - Crear un **hilo secundario**: más trabajo pero permite mayor flexibilidad

IntentService

- Declaración (la mayor parte del trabajo lo hace la clase IntentService):

```
public class MyIntentService extends IntentService {

    public MyIntentService() {
        super("MyIntentService");
    }

    @Override
    protected void onHandleIntent(Intent intent) {
        if (intent != null) {
            String action = intent.getAction();
            Log.d ("_TAG", "inicio action "+action);
            // tarea pesada
            Log.d ("_TAG", "ejecutada action "+action);
        }
    }
}
```

IntentService

- Se pueden lanzar diferentes instancias de un IntentService
  - Pero el sistema las ejecutará de una en una
  - Cada instancia (intent) puede tener su propia Action y datos

```
Intent intentService = new Intent(getApplicationContext(), MyIntentService.class);
intentService.setAction("my_action1");
startService(intentService);
intentService.setAction("my_action2");
startService(intentService);
intentService.setAction("my_action3");
startService(intentService);
```

Índice

1. Receptores de emisiones
2. Servicios
3. **Comunicaciones con servicios**
4. Tareas en segundo plano

Comunicaciones con servicios

- *Intent* de lanzamiento: comunicación unidireccional hacia el servicio
- Receptor: el servicio emitir una emisión (puede ser local) a la que se ha suscrito el componente que lo ha ejecutado. Comunicación unidireccional desde el servicio
- Enlace (*IBinder*): cuando el servicio se ejecuta en el mismo proceso que su lanzador (app). Comunicación bidireccional síncrona (simple, rápida y eficiente)
- Paso de mensajes: comunicación (bidireccional) asíncrona (típica de los hilos)
- AIDL: interfaz válida para comunicaciones entre diferentes procesos (distintas apps)

Servicios enlazados

- Un servicio enlazado o vinculado permite que los componentes se vinculen con él, envíen solicitudes, reciban respuestas y establezcan comunicaciones entre procesos
  - Sólo está activa mientras presta servicio a otro componente de la app
  - No se ejecuta en segundo plano indefinidamente
- La vinculación se realiza con un IBinder que define la interfaz de comunicación
  - El *IBinder* debe ser declarado y extendido
  - Proceso asíncrono que debe implementar la interfaz *ServiceConnection*
  - Y gestionar los eventos *onServiceConnected*() y *onServiceDisconnected*()

Servicios enlazados: ejemplo declaración

```
public class MyBoundService extends Service {
    // Random number generator
    private final Random mGenerator = new Random();
    /** methods for clients */
    public int getRandomNumber() {
        return mGenerator.nextInt(100);
    }
    // Binder given to clients
    private final IBinder binder = new MyBinder();
    public class MyBinder extends Binder {
        MyBoundService getService() {
            // Return this instance of LocalService so clients can call public methods
            return MyBoundService.this;
        }
    }
    @Override
    public IBinder onBind(Intent intent) {
        return binder;
    }
}
```



Servicios enlazados: ejemplo uso (I)

```
public class MainActivity extends AppCompatActivity {

    MyBoundService myBoundService;
    Boolean mBound = false;

    [...]

    but_rnd.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            // Always check if there is a valid connection
            if (mBound) {
                // Methods can be in both directions (get and set)
                int num = myBoundService.getRandomNumber();
                tv_hello.setText("Random number: " + num);
            }
        }
    });
```

Servicios enlazados: ejemplo uso (II)

```
@Override
protected void onStart() {
   super.onStart();
   // Bind to the Bound Service  with an explicit Intent
   Intent intent = new Intent(getApplicationContext(), MyBoundService.class);
   bindService(intent, connection, Context.BIND_AUTO_CREATE);
}

@Override
protected void onStop() {
   super.onStop();
   // Unbind from the Bound Service and update the connection flag
   unbindService(connection);
   mBound = false;
}
```

Servicios enlazados: ejemplo uso (y II)

```
/** Defines callbacks for service binding, passed to bindService() */
private ServiceConnection connection = new ServiceConnection() {

    @Override
    public void onServiceConnected(ComponentName className,
                                   IBinder service) {
        // We've bound to LocalService, cast the IBinder and get LocalService instance
        MyBoundService.MyBinder binder = (MyBoundService.MyBinder) service;
        myBoundService = binder.getService();
        mBound = true;
    }

    @Override
    public void onServiceDisconnected(ComponentName arg0) {
        mBound = false;
    }
};
```

Índice

1. Receptores de emisiones
2. Servicios
3. Comunicaciones con servicios
4. **Tareas en segundo plano**

Tareas en segundo plano

- Las tareas en segundo plano son una parte fundamental de las app
- Principio rector: toda tarea que requiera más de unos pocos milisegundos para su ejecución debe delegarse a un subproceso en segundo plano
  - Actualización base de datos, acceso a ficheros o a la red, decodificación, etc.
- Android impone cada vez más restricciones a su ejecución
  - Se aducen problemas de eficacia energética: ahorro de batería
  - Gestionar los permisos de las apps (Optimización de la batería)
  - Ojo: hay fabricantes más restrictivos con las apps que pasan a segundo plano

Tareas en segundo plano: categorías

- Tarea inmediata: si debe completarse mientras el usuario interactúa con la app
  - Servicios en primer plano (=> se verán en el tema de las notificaciones)
  - Corutinas Kotlin o subprocesos (hilos) en Java (complejos)
  - WorkManager (funciona aunque la app pase a segundo plano)
- Tarea exacta: si debe ejecutarse en un momento preciso
  - AlarmManager
- Tareas diferidas: el resto
  - IntentService (pero sin acceso a la UI)
  - AsyncTask (acceso a la UI, pero no gestiona errores ni cambios de configuración)
  - WorkManager (la mejor opción: gestiona el reinicio del dispositivo)

Tareas en segundo plano: WorkManager

- Librería sencilla pero muy flexible y potente:
  - Compatible con tareas asíncronas únicas y periódicas
  - Admite restricciones en cada tarea (estado red, almacenamiento o carga)
  - Permite encadenar solicitudes de trabajo complejas y ejecución en paralelo
  - Funciona con y sin los servicios de Google Play
  - Sigue las recomendaciones de seguridad del sistema
  - Es compatible con LiveData para mostrar el estado de la solicitud en la IU
  - Según la versión de Android, usa otras APIS: JobScheduler y AlarmManager

WorkManager: ejemplo declaración

- Definir la subclase que realizará el trabajo:

```
public class UploadWorker extends Worker {
    public UploadWorker(
            @NonNull Context context,
            @NonNull WorkerParameters params) {
        super(context, params);
    }

    @Override
    public Result doWork() {
        // Do the work here--in this case, upload the images.

        // Indicate whether the work finished successfully with the
Result
        return Result.success();
        // Other options: Result.failure() and Result.retry()
    }
}
```

WorkManager: ejemplo uso

- No olvidar declarar las dependencias en el fichero build.gradle de la app:

```
implementation 'androidx.work:work-runtime:2.6.0'
```

- Programar el trabajo y enviar al sistema para su ejecución:

```
WorkRequest uploadWorkRequest =
       new OneTimeWorkRequest.Builder(UploadWorker.class).build();

WorkManager.getInstance(this).enqueue(uploadWorkRequest);
```

- Se puede cancelar un trabajo enviado (por id, tag o nombre) o todos los trabajos:

```
WorkManager.getInstance(this).cancelWorkById(uploadWorkRequest.getId());
WorkManager.getInstance(this).cancelAllWork();
```

WorkManager otras opciones

- Puede ejecutar tanto tareas única como periódicas (enqueueUniquePeriodicWork)
- Se pueden enviar y recibir datos
- Se pueden definir restricciones con setConstraints y Constraints.Builder:
  - setRequiredNetworkType, setRequiresBatteryNotLow, setRequiresCharging, ...
- Integra mecanismos de resolución de conflictos (REPLACE, KEEP y APPEND)
- Se puede verificar el estado de la tarea enviada en cualquier momento y también registrar observables para ser informados de los cambios
- También se puede observar el progreso intermedio de un trabajo

WorkManager: envío y recepción datos

- Para enviar datos al trabajo antes de enviarlo:

```
Data data = new Data.Builder().putString(UploadWorker.KEY_DATA, url).build();
WorkRequest uploadWorkRequest = new OneTimeWorkRequest
                    .Builder(UploadWorker.class)
                    .setInputData(data).build();
```

- Para recoger los datos y devolver los resultados dentro del método doWork():

```
String url = getInputData().getString(KEY_DATA);

  // do the task
Data data = new Data.Builder().putString(KEY_IMAGE_URI,
outputUri.toString()).build();
return Result.success(data);
```

AsyncTask

- AsyncTask puede ejecutar tareas en segundo plano
- Ventajas:
  - Relativamente simple y fácil de programar, ampliamente usado
  - Puede modificar la UI (al principio, durante y al final de la tarea)
  - Puede ir mostrando el progreso de la tarea
- Inconvenientes:
  - Sólo se ejecutan de una en una (hay que chequearlo)
  - No se pueden modificar los parámetros
  - No tiene una buena gestión de errores
  - Problemas al cambiar la configuración (UI) y fallos de memoria (memory leaks)
  - Está *deprecated*

AsyncTask

- AsyncTask se declara con 3 parámetros: entrada, progreso y salida
  - Cada parámetro puede ser: void, un único tipo o un array de un tipo
- Sólo tiene un método obligatorio (trabajo en background): doInBackground
- Opcionalmente, se pueden definir otros 3 métodos (típicamente para cambiar la UI)
  - onPreExecute: se llama antes de ejecutar la tarea
  - onProgressUpdate: durante la ejecución de la tarea (usando publishProgress en el método principal de doInBackground)
  - onPostExecute: se llama al finalizar la tarea de forma correcta
- La tarea se puede cancelar y entonces se ejecuta onCancelled
  - Durante la ejecución se debe comprobar que no se ha cancelado (isCancelled)

AsyncTask

```
public class CountTask extends AsyncTask<Integer, Integer, String> {
   protected String doInBackground(Integer... myList) {
       Log.d(TAG, "Parameters: " + myList.toString());
       for (int i = 0; i < myList.length; i++) {
           try {
               publishProgress((int) ((i / (float)myList.length) * 100));
               Thread.sleep(5000);                // Simulate a long task
           } catch (InterruptedException e) {
               e.printStackTrace();    Thread.currentThread().interrupt();
           }
           // Escape early if cancel() is called
           if (isCancelled()) break;
       }
       return "Done ";
   }
   protected void onPreExecute() {   Log.d(TAG, " Init ... " );   }
   protected void onProgressUpdate(Integer... progress) { Log.d(TAG, " "+ progress[0]); }
   protected void onPostExecute(String result){ Log.d(TAG, "Finished " ; }
   protected void onCancelled() {    Log.d(TAG, "Cancelled " );  }
}
```

AsyncTask

- Para ejecutar una nueva tarea:

```
CountTask myCountTask  = new CountTask();
myCountTask.execute(10,6,7);
```

- Pero atención, sólo se puede ejecutar una instancia de cada vez
  - Antes de relanzar una tarea se debe comprobar que ha acabado
- También se pueden lanzar varias tareas en paralelo:

```
myCountTask.executeOnExecutor(AsyncTask.THREAD_POOL_EXECUTOR,5,6,6,7,7);
```

- Para cancelar una tarea:

```
myCountTask.cancel(true);
```

Hilos

- En Android se pueden usar hilos
- Ventajas:
  - Verdadera concurrencia y paralelismo
  - Paso de mensajes para comunicaciones bidireccionales: solución general
- Inconvenientes:
  - Relativamente complejo
  - Desde un hilo secundario no se puede acceder a la UI
  - No tiene una buena gestión de errores (no reusar hilos: crear de cada vez)
- Siempre deben finalizar o ser interrumpidos (evitar anónimos, guardar referencias)
  - Problemas al cambiar configuración (UI): poner en un servicio (local o enlazado)

Hilos: declaración

```
class CountThread extends Thread {
   private final int count;
   CountThread(int count) {
       this.count = count;
   }

   public void run() {
       // tarea pesada
       for (int i = 0; i < count; i++) {
           try {
               Thread.sleep(5000);
           } catch (InterruptedException e) {
               return;
           }
       }
   }
}
```

```
Runnable CountRun = new Runnable() {

   @Override
   public void run() {
       int count = 10;
       // tarea pesada
       for (int i = 0; i < count; i++) {
           try {
               Thread.sleep(5000);
           } catch (InterruptedException e) {
               return;
           }
       }
   }
};
```

```
Thread countThread = new CountThread(10);
countThread.start();
countThread.interrupt();
```

```
Thread countRun = new Thread(CountRun);
countRun.start();
countRun.interrupt();
```

Hilos

- Los hilos secundario no pueden acceder directamente a la UI
  - Se puede usar el UIHandler del hilo principal (o UI)
  - O más fácil con View.post(Runnable) o Activity.runOnUiThread(Runnable)
    - Implica una dependencia con la vista
- Pero lo recomendable es desacoplar estado/datos de la vista

```
private void showData (String data) {
    tv_output.post(new Runnable() {
        @Override
        public void run() {
            tv_output.setText(data);
        }
    });
```

Referencias

- https://developer.android.com/guide/components/broadcasts
- https://developer.android.com/reference/android/content/BroadcastReceiver
- https://developer.android.com/about/versions/11/reference/broadcast-intents-30
- https://developer.android.com/guide/components/services
- https://developer.android.com/reference/android/app/Service
- https://developer.android.com/topic/libraries/architecture/workmanager/basics
- https://developer.android.com/topic/libraries/architecture/workmanager/how-to/define-work
- https://developer.android.com/reference/android/os/AsyncTask
- https://developer.android.com/reference/java/lang/Thread

